

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Web-System Architecture of Your Own Project

Web-System Architectures and End-to-End Data Flow

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 2
Output	System-specific architecture diagram, request-to-response trace, and group presentation
Maximum Score	80 points

Student / Project Information

Block: A1	Date: 09/10/26
Group No.:	Instructor: Jean Y. Villasin, MIT
System Title: Seaweed (Guso) Quality Classification Using Image Processing	
Members: ZYDANE ANGELO ADANZA JHERICK PEÑALOSA TIMOTHY JOSUA TAVERA	

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain common web-system architectures and trace the end-to-end flow of data through major system components.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Web-system architecture diagram and request-to-response tracing exercise.

Purpose of the Activity

Before writing major code, your team must create an architecture blueprint that explains how the users, Flutter interface, application or API layer, database, external services, security controls, and deployment environment work together. The diagram will serve as a technical guide throughout the semester project.

Important distinction: Flutter is the client-side application framework. It is one part of the complete web system; your architecture must also show how data is processed, stored, secured, and returned to the user.

2. Intended Learning Objectives

- Identify the users, components, layers, services, and deployment resources required by the team's proposed system.
- Select and justify an appropriate architecture such as two-tier, three-tier, monolithic, service-oriented, or a defensible hybrid architecture.
- Create a clear system-specific architecture diagram using accurate component names, directional arrows, protocols, and data labels.
- Trace one important workflow from the user action through the Flutter client, backend/API, business logic, data store, and response shown to the user.

3. Activity Specifications

Mode	Existing semester-project group; all members must participate and be able to explain the diagram.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One system architecture diagram based on the team's actual approved project.
Supporting Output	One request-to-response trace, a 100–150 word architecture justification, and a group presentation.
Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, or an equivalent diagramming tool.
Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned architecture component or data-flow step.

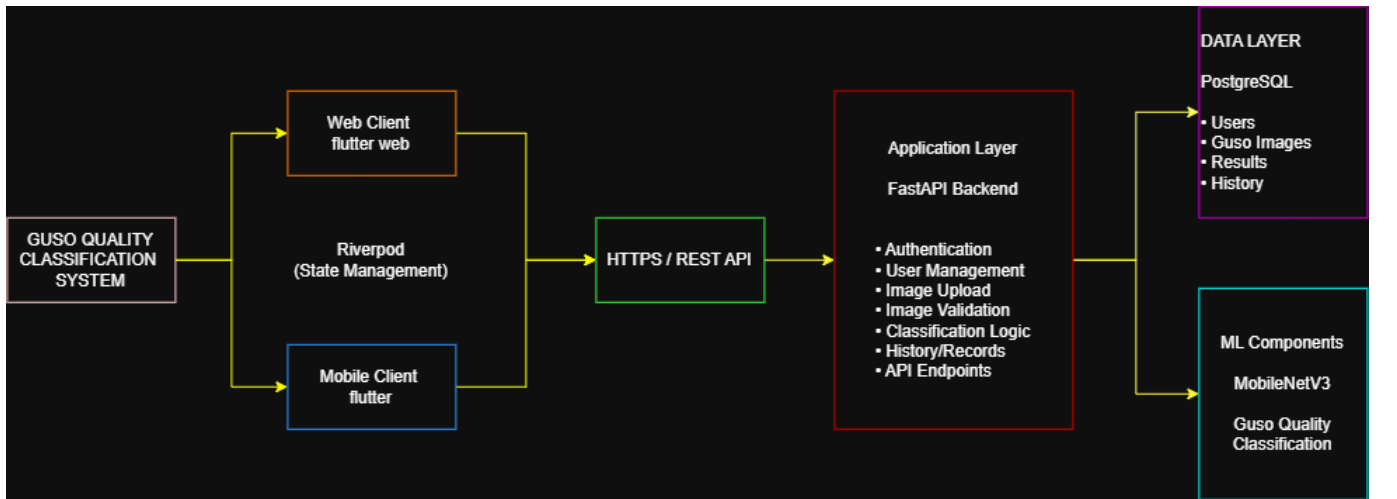
4. Pre-Laboratory Requirements

- Approved or proposed system title and a brief statement of the community or organizational problem being addressed.
- List of primary users or roles and at least three major features of the system.
- Tentative technology stack, including Flutter, backend/API or backend-as-a-service, database, external services, and deployment platform.
- A copy of the lecture notes on web-system layers, two-tier and three-tier architectures, monolithic and service-oriented architectures, and request-response flow.

5. ✓ Part A – System Component Inventory

Complete the worksheet before drawing. Use the exact names planned for your own project. Do not write only generic labels such as “front end” or “database” when a more specific component name is already known.

Architecture Item	Your System	Notes / Examples of Evidence
System title and purpose	Seaweed (Guso) Quality Classification Using Image Processing	The system helps farmers and sellers classify the quality of Guso using uploaded images and a MobileNetV3 machine learning model.
Actors / user roles	Farmers / Seller / Seaweed Classifier	Farmers and sellers can log in, upload Guso images, view classification results, and access their history/records.
Flutter client	Flutter Web	Provides the web interface, including Login/Register, Dashboard, Guso Quality Classification, History/Records, and Download/Save Record.
State and client logic	Riverpod	Manages application state and user actions in the Flutter Web client. Routing/navigation and client-side form/image validation are also handled on the client side.
Backend / API / BaaS	FastAPI REST API	Provides REST API endpoints and handles requests from Flutter Web, authentication, authorization, validation, and system processing.
Business logic	Image validation, image processing, Guso quality classification	Processes the uploaded Guso image, performs image preprocessing, uses the MobileNetV3 model for classification, and manages classification records/history.
Database and storage	PostgreSQL	Stores user accounts, Guso images, classification results, and history/records.
External services	None	No separate external service is currently used. The MobileNetV3 model is integrated directly into the FastAPI backend rather than accessed as an external service.
Security controls	Authentication, input/image validation, authorization	Protects user accounts and system resources through login verification, token checks, role checking, request validation, and image validation.



6. ✓ Part B – Select the Architecture Style

Select the architecture that best fits your system. More than one may apply, but the final choice must be justified based on requirements—not merely because it is familiar.

Architecture	Core Structure	When It May Fit	Select
Two-tier	Client communicates directly with a data service or tightly coupled server.	Small systems with controlled scope; security and maintainability must still be addressed.	[]
Three-tier	Flutter presentation → application/API → database.	Recommended for systems that need clear separation of responsibilities and secure data access.	[✓]

Architecture	Core Structure	When It May Fit	Select
Monolithic	Application modules are packaged and deployed as one backend application.	Small to medium systems requiring simpler deployment and coordinated modules.	[]
Service-oriented / Hybrid	Multiple services, BaaS modules, or external APIs cooperate.	Systems that genuinely need independent services such as GIS, ML, notifications, or payments.	[]

Selected architecture: Tier Three

Initial justification (3–5 sentences):

We selected the Three-Tier Architecture because our system separates the presentation layer using Flutter and Flutter Web, the application layer using FastAPI for API and business logic, and the data layer using PostgreSQL. This separation makes the system easier to manage, maintain, and scale while allowing both the mobile and web clients to use the same backend.

7. ✓ Part C – Construct the Web-System Architecture Diagram

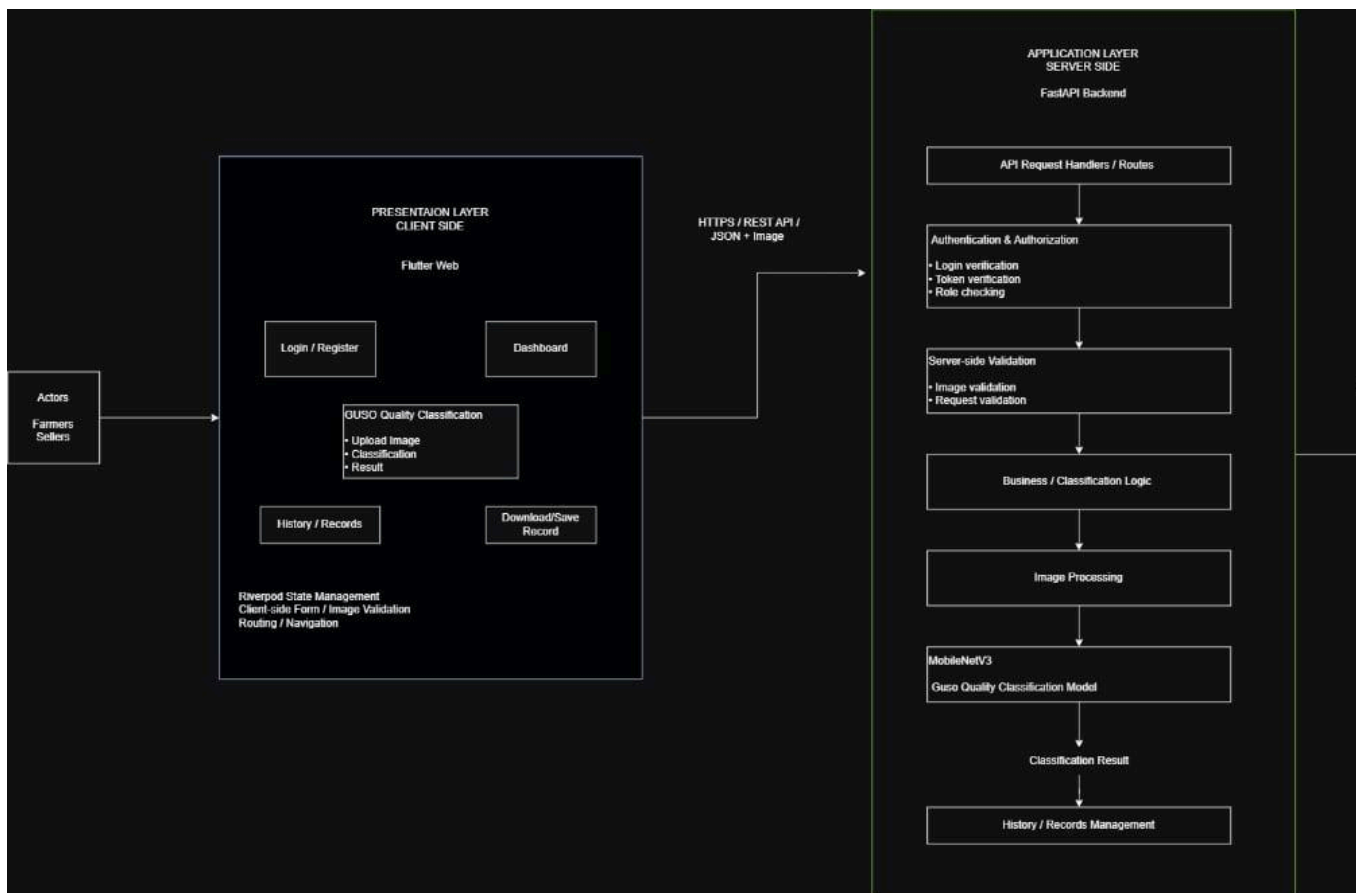
Create one clean, readable diagram that shows the complete architecture of your own system. The diagram must be understandable even to a reviewer who has not seen your code.

No.	Required Element	What Must Be Shown
1	System identity	Place the official system title and selected architecture style at the top.
2	Actors	Show every major user role or external actor that sends requests to the system.
3	Client / presentation layer	Show Flutter Web or Flutter mobile, key screens or UI modules, routing, state management, and form validation as appropriate.
4	Communication layer	Show how components communicate: HTTPS, REST, WebSocket, SDK, JSON, or another relevant mechanism.
5	Application / API layer	Show request handlers, authentication, authorization, validation, service/business logic, and major server modules.
6	Data layer	Show the selected database, important tables/entities or data domains, file storage, relationships, and constraints at an appropriate level.
7	External services	Show only services actually used by the project, such as GIS/maps, ML model or API, email/SMS, payment, weather, or cloud storage.
8	Deployment	Show where the Flutter client, backend/API, database, and storage will run or be hosted.
9	Data flow	Use directional arrows and label important requests, responses, protocols, data formats,

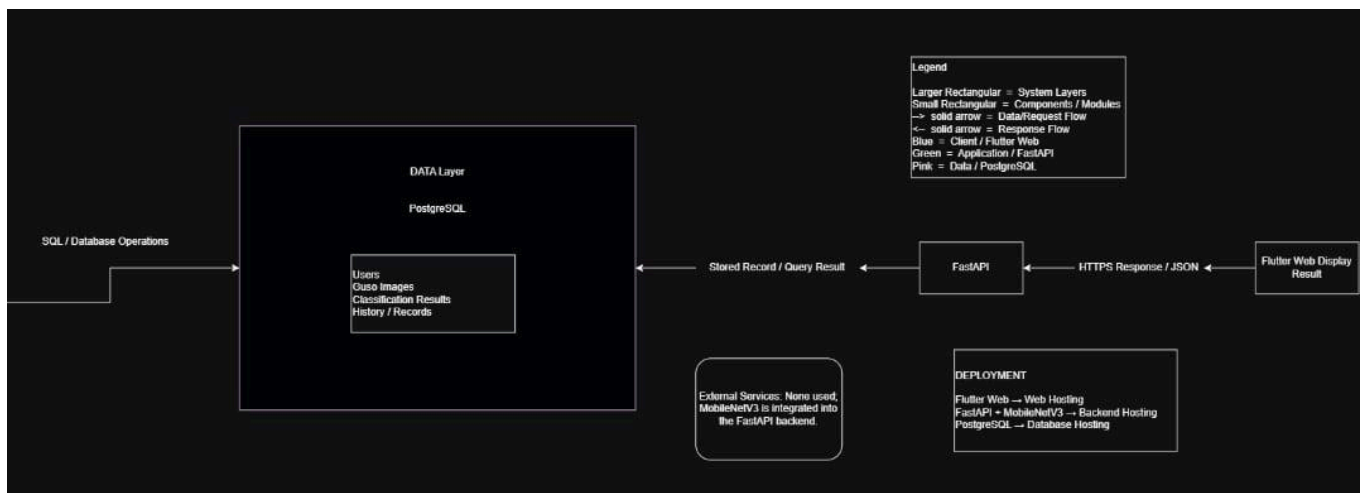
		and returned results.
10	Security	Show where login, role checks, tokens, input validation, access control, and protected resources are handled.
11	Legend	Use consistent symbols, colors, line styles, and a brief legend.

Minimum Diagram Rule

Your architecture must show at least one complete path: Actor → Flutter interface → API/application component → database or external service → response returned to the Flutter interface. Every arrow on this path must have a meaningful label.



Continuation



8. Reference Example – Do Not Copy as Your Final Output

The example below illustrates the level of clarity expected in a Flutter-based architecture. It is not a template to copy. Replace it with the actual users, features, services, data, security controls, and deployment choices of your own system.

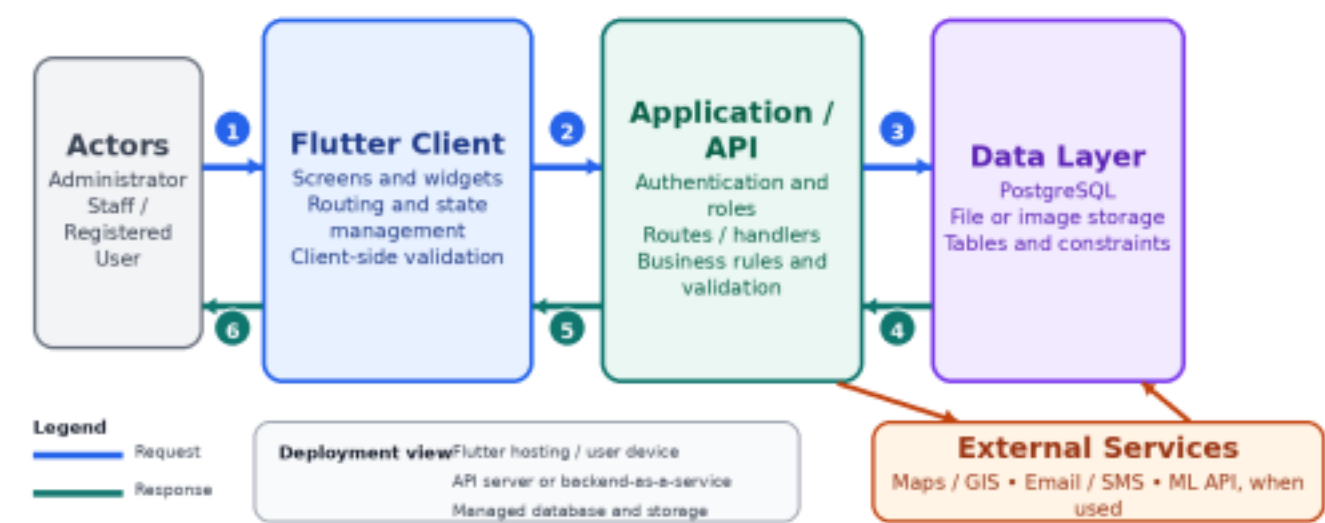


Figure 1. Example of a complete request-and-response path in a Flutter-based three-tier architecture.

1. User action	2. HTTPS / JSON request	3. Query or transaction
4. Records or result	5. HTTP status + JSON	6. Updated Flutter interface

9. Recommended Architecture Labels for Flutter Projects

Layer / Area	Possible Label	What It Represents
Presentation	Flutter Web / Flutter App	Screens, widgets, forms, navigation, and user feedback
Client state	Provider / Riverpod / BLoC / controller	Coordinates UI state and user actions
Communication	HTTP client, SDK, or repository/service class	Sends requests and receives responses
Application	REST API, BaaS, serverless function, or custom backend	Routes requests, validates data, and applies business rules
Security	Auth service, token verification, role checks	Protects accounts, data, and restricted operations
Data	PostgreSQL, storage, and important entities	Persists system records and files

10. Laboratory Procedure

Phase 1 – Analyze the system (20 minutes)

Review the approved title, problem, users, major features, and available technology choices. Complete the component inventory.

Phase 2 – Choose the architecture (20 minutes)

Select the architecture style and write an initial justification based on security, maintainability, scalability, integration, and deployment.

Phase 3 – Draft the diagram (45 minutes)

Arrange the actors and layers from left to right or top to bottom. Add all major system components before

drawing arrows.

Phase 4 – Label data flow (35 minutes)

Label requests and responses using actual workflows, protocols, endpoints or service names, data formats, status/result, and security checks.

Phase 5 – Trace one key transaction (30 minutes)

Use the table in Part D to trace one important user action through the complete architecture.

Phase 6 – Review and improve (15 minutes)

Use the quality checklist. Ask another group or the instructor to identify missing components, unclear arrows, or incorrect layer responsibilities.

Phase 7 – Export and submit (15 minutes)

Export the final diagram, complete the justification, commit the files to GitHub, and submit the required files through MasaoLMS.

11. ✓ Part D – Request-to-Response Tracing Exercise

Select one critical workflow from your own system. Examples include login, submitting a report, registering a boarding house, approving a request, uploading an image, requesting a prediction, or retrieving a map-based recommendation.

Selected workflow:

Uploading a Guso image for quality classification

Expected final result shown to the user:

The classified quality result (like "Grade A / Good Quality") displayed on the Guso Quality Classification screen, along with the image and a saved record in History.

Step	Actor / Component	Action or Data	Protocol / Mechanism	Expected Result
1	Farmer/Seller (Actor)	Selects/captures a Guso image and taps "Classify"	User interaction (UI tap)	Image selected and ready for upload
2	Flutter Web (Presentation Layer)	Validates image format/size before sending; Riverpod updates local state to "loading"	Client-side validation	Only valid images proceed; UI shows loading indicator
3	Flutter Web → FastAPI	Constructs and sends the request with image file and auth token	HTTPS POST / REST API (JSON + Image, multipart)	Request received by backend
4	FastAPI – API Request Handler	Routes the incoming request to the correct endpoint (e.g., /classify)	REST routing	Request forwarded to Authentication module
5	FastAPI – Authentication & Authorization	Verifies login token and checks user role	Token/JWT verification	Authorized requests proceed; unauthorized ones return 401
6	FastAPI – Server-side Validation	Re-validates the image (format, size, corruption checks) and request data	Server-side validation	Valid image passed to business logic; invalid image returns error
7	FastAPI – Business/Classification Logic → Image Processing	Preprocesses the image (resize, normalize) for the model	Internal function call	Image converted to model-ready input
8	MobileNetV3 (ML Component)	Runs classification on the preprocessed image	Model inference (in-process, integrated into FastAPI)	Classification result generated (e.g., quality grade + confidence score)

9	FastAPI – History/Records Management → PostgreSQL	Saves the image reference, classification result, and timestamp to the database	SQL / Database Operations (INSERT)	Record stored in Guso Images, Classification Results, and History tables
10	FastAPI → Flutter Web (Response)	Sends classification result back to the client; Riverpod updates state; UI displays result	HTTPS Response / JSON	User sees the classification result on screen; record appears in History

Tracing Guidance

A strong trace normally includes user input, Flutter-side validation, construction of the request, authentication or token checking, server-side validation, business-rule execution, database or external-service access, response generation, Flutter state update, and user feedback.

12. ✓ Part E – Architecture Justification

Write 100–150 words explaining why the selected architecture is appropriate for your system. Address all of the following:

- How the architecture supports the system’s main users and features.
- Why are the responsibilities divided among the Flutter client, application/API layer, and data layer.
- How the design supports security, maintainability, scalability, integration, and deployment.
- Any limitation or risk that the team must manage during implementation.

The Three-Tier architecture effectively supports the Seaweed Guso Quality Classification System by enforcing a strict separation of responsibilities. The presentation layer utilizes Flutter and Flutter Web, providing users with a seamless, cross-platform interface for capturing and uploading seaweed images, while Riverpod efficiently manages local client state. The application layer, built with FastAPI, centralizes all business logic. This allows the backend to handle authentication, securely process REST API requests, and seamlessly bridge data with the MobileNetV3 machine learning component. By isolating the PostgreSQL database in the data layer, we ensure secure, structured storage of user credentials and classification history. This decoupled design enhances scalability, allowing the resource-intensive ML model and backend to scale independently from the client interfaces. A primary risk the team must manage is the processing latency between the FastAPI backend and the MobileNetV3 model to prevent HTTP connection timeouts during image evaluation.

13. Final Quality Checklist

☐ The diagram uses the team's official system title and actual project features.	☐ Deployment locations or platforms are identified.
☐ The selected architecture style is clearly named and justified.	☐ Request and response arrows have clear directions and meaningful labels.
☐ All major actors or user roles are present.	☐ At least one complete request-to-response path is visible.
☐ The Flutter client and important UI/client responsibilities are shown.	☐ Authentication, authorization, validation, and protected resources are represented.
☐ The application/API or backend-as-a-service components are shown.	☐ The diagram has a legend, readable text, consistent symbols, and no crossing arrows that cause confusion.
☐ The database, file storage, and important data domains are shown.	☐ The request-to-response table agrees with the diagram.

[] External services are included only when the project actually uses them.

[] Every group member can explain an assigned component, trace the complete workflow, and answer questions without reading a script.

14. Submission Requirements

- PDF or PNG of the final architecture diagram in readable landscape orientation.
- Editable diagram source file, such as .drawio, .pptx, .fig, or the native file of the selected tool.
- Completed request-to-response tracing table and 100–150 word architecture justification.
- Files committed to the project GitHub repository under a folder such as docs/architecture/.
- Repository link and required files submitted through MasaoLMS using the naming convention: IT110_LA2_Group##_SystemName.
- Deliver a 5–7 minute group presentation using the final architecture diagram as the main visual, followed by a 2–3 minute question-and-answer period. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own system and design decisions. AI tools may be used to clarify concepts, but the final architecture, labels, request flow, and justification must be reviewed, understood, and defended by the team. The instructor may ask any member to explain a component or trace a request without assistance.

15. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate.

For the presentation criterion, points may be reduced for unclear explanation, unequal participation, excessive reading, incorrect tracing, poor time management, or inability to answer questions about the team's own design.

Criterion	Evidence for Full Credit	Points
Architecture correctness and selection	The selected architecture is appropriate for the system requirements, accurately represented, and justified using clear architectural reasoning.	15
System-specific components and layers	Actors, Flutter client, application/API, business logic, data layer, external services, security controls, and deployment are complete and use the team's actual project labels.	12
Request-to-response and data flow accuracy	Arrows, protocols, request/response direction, validation, processing, data access, and returned results form a complete, consistent, and technically coherent workflow.	15
Security and data protection	Authentication, authorization, role checks, input validation, protected resources, data access controls, and appropriate trust boundaries are clearly represented.	8
Technology-stack integration and deployment	The diagram clearly shows how Flutter, the selected backend/API or BaaS, database, storage, external services, and deployment platforms work together.	8
Visual clarity and technical notation	The diagram uses readable text, consistent symbols, meaningful labels, a useful legend, balanced spacing, and directional arrows with minimal visual confusion.	7
Written architecture justification	The 100–150 word explanation clearly connects the selected architecture to the system's users, features, security, maintainability, scalability, integration, deployment, limitations, and risks.	5
Group presentation and oral defense	The presentation is clear, organized, and completed within the assigned time; the final diagram is used effectively; every member participates; the group accurately explains the architecture and request-to-response trace;	10

	and questions are answered correctly and confidently.	
--	---	--

Raw Score	/ 80	Equivalent	_____
Checked by	_____	Date	_____
Remarks			

16. Post-Laboratory Reflection

Which component should contain most of the business rules in your system, and why?

The FastAPI application layer should contain most of the business rules, since it centralizes authentication, validation, image preprocessing, and the classification logic that MobileNetV3 calls. Keeping these rules on the server (rather than in Flutter) ensures consistency across web and mobile clients, prevents users from bypassing validation or classification logic, and keeps the resource-intensive ML processing away from the client device.

What should the system do when the API, database, or internet connection becomes unavailable?

The Flutter client should detect the failure (timeout or connection error) and show a clear, user-friendly message rather than freezing or crashing for example, "Unable to connect. Please check your internet connection and try again." Any in-progress image upload should be cancelled safely, and Riverpod should reset the UI state so the user can retry. If the database specifically is unavailable, FastAPI should return a proper error response (like the HTTP 503) instead of exposing raw error stack traces, and the classification result should not be marked as saved until the record is confirmed in PostgreSQL.

Which part of the architecture is most security-sensitive?

The Authentication & Authorization module in the FastAPI application layer is the most security-sensitive part, since it controls login verification, token checking, and role-based access to protected resources like image uploads and classification history. This is the main trust boundary between unauthenticated actors and the system's data and business logic, so weaknesses here (like weak token validation) could expose user accounts or classification records.

How would the architecture change if the number of users or stored records increased significantly?

The backend and database would need to scale independently of the client, which the three-tier design already supports. Practically, this could mean adding caching, connection pooling, or read replicas for PostgreSQL, deploying multiple FastAPI instances behind a load balancer, and possibly moving MobileNetV3 inference to a separate, independently scalable service (rather than being integrated directly in FastAPI) if classification requests become a bottleneck. Image storage might also need to move to dedicated object storage instead of growing indefinitely inside the database.

Which component or feature should your team implement first after this activity?

The team should implement the Authentication & Authorization module first, since nearly every other feature (image upload, classification, history) depends on knowing who the user is and what they're allowed to do. Right after that, the core image upload → classification pipeline (FastAPI + MobileNetV3 integration) should be built next, since it's the system's primary value-delivering feature.